Лабораторная работа №3

Асимметричная криптография. Криптосистема Рабина

Пояснительная записка к защите. Реализация: internal/crypto/rabin, язык Go

---

Содержание

1. Что сделано в работе 2
2. Обозначения 2
3. Паспорт схемы 2
4. Тонкие моменты 3
   - 4.1. Простота и остаток – два разных условия 3
   - 4.2. Почему нужно именно $p \equiv 3 \pmod 4$ 3
   - 4.3. Почему `p.Bit(1) == 1` равносильно $p \equiv 3 \pmod 4$ 3
   - 4.4. Четыре корня и китайская теорема об остатках 4
   - 4.5. Как из четырёх корней выбирается истинный 4
   - 4.6. Зачем блок короче модуля и зачем маркер 4
   - 4.7. Детерминированность и её следствия 5
   - 4.8. Почему $p \neq q$ 5
   - 4.9. Размеры и расширение 5
5. Реализация 5
   - 5.1. Структура проекта 5
   - 5.2. Формат ключей 6
   - 5.3. Порядок работы дешифратора 6
   - 5.4. Интерфейс 6
6. Проверка работоспособности 6
7. Вопросы на защите 6

1. Что сделано в работе

Реализована криптосистема Рабина: генерация ключевой пары, зашифрование возведением в квадрат по модулю n и расшифрование через квадратные корни по модулю простых чисел и китайскую теорему об остатках. Неоднозначность из четырёх корней устранена избыточностью в блоке, поэтому дешифратор возвращает единственный открытый текст. Схема встроена в терминальный интерфейс отдельной вкладкой.

Часть	Что	Функции Go	Проверка
ключи	простые $p, q \equiv 3 \pmod 4$, модуль $n = pq$	`GenerateKeys`, `randPrime3mod4`	размер модуля, $p \neq q$
зашифрование	разбиение на блоки, избыточность, $c = m^2 \bmod n$	`Encrypt`, `pad`, `chunk`	круговой тест
расшифрование	корни по модулю p и q, CRT, выбор корня	`Decrypt`, `Roots`, `sqrtModPrime`, `recoverPayload`	пример $p = 7$, $q = 11$; ровно один корень с избыточностью
интерфейс	вкладка `Rabin`, ручной ввод и файлы	`ui/asympane`	сборка, ручная проверка

> **Ключевые числа для защиты**
> Модуль – **512, 1024 или 2048 бит**. Блок – на **один байт короче** модуля (63, 127 или 255 байт). В блоке **9 байт** служебных: маркер `0x80` и метка `MZIRABIN`. Полезная нагрузка – 54, 118 или 246 байт соответственно. Корней у шифртекста – **четыре**, избыточность проходит ровно один.

2. Обозначения

Запись	Смысл
p, q	Простые числа закрытого ключа, оба сравнимы с 3 по модулю 4 и не равны друг другу.
n	Открытый ключ: $n = pq$. Модуль, по которому ведутся все вычисления.
m	Открытый текст как число, $0 \le m < n$. В коде – блок фиксированной длины.
c	Шифртекст одного блока: $c = m^2 \bmod n$.
m_p, m_q	Квадратные корни из c по модулю p и по модулю q.
y_p, y_q	Коэффициенты Безу: $y_p p + y_q q = 1$. В коде их возвращает `GCD`.
r, s	Два корня, собранные по CRT; ещё два получаются сменой знака: $n - r$ и $n - s$.
`k`	Длина модуля в байтах: `byteLen(n)`. Шифртекст каждого блока записывается ровно в `k` байт.
`blockLen`	Длина блока: `k - 1`. В блоке всегда на байт меньше, чем в модуле, чтобы любое значение было меньше n.

3. Паспорт схемы

Параметр	Значение
Год	1979, схема предложена Михаэлем Рабином
Тип	асимметричная криптосистема с открытым ключом
Стойкость	эквивалентна факторизации n: извлечение корней по модулю составного числа сводится к разложению n на множители

Открытый ключ	$n = pq$
Закрытый ключ	пара (p, q)
Зашифрование	одно возведение в квадрат: $c = m^2 \bmod n$
Расшифрование	два возведения в степень по модулю простых чисел, CRT, перебор четырёх корней
Неоднозначность	четыре корня; снимается избыточностью
Скорость	зашифрование быстрее RSA при той же длине ключа; расшифрование сравнимо

4. Тонкие моменты

4.1. Простота и остаток – два разных условия

Фраза «выбираются простые числа p и q, удовлетворяющие условию $p \equiv q \equiv 3 \pmod 4$» содержит **два независимых требования**:

1. число должно быть простым;
2. остаток от деления на 4 должен быть равен 3.

Запись $p \equiv 3 \pmod 4$ сама по себе простоты не гарантирует. Она говорит только о младших двух битах числа:

- 7 – простое, $7 \bmod 4 = 3$: подходит;
- 11 – простое, $11 \bmod 4 = 3$: подходит;
- $15 \bmod 4 = 3$, но $15 = 3 \cdot 5$: не подходит, хотя условие на остаток выполнено.

Поэтому в коде сначала вызывается `rand.Prime`, который гарантирует простоту, и только потом отбрасываются числа с неподходящим остатком. Проверять остаток вместо простоты нельзя: под условие $p \equiv 3 \pmod 4$ попадает бесконечно много составных чисел.

> **Почему среди простых чисел подходит примерно половина**
> Любое простое число, кроме двойки, нечётное, поэтому его остаток по модулю 4 равен либо 1, либо 3. Простые числа распределены между этими классами примерно поровну, так что цикл генерации в среднем делает пару попыток.

4.2. Почему нужно именно $p \equiv 3 \pmod 4$

Для простого p квадратный корень из c можно вычислить по формуле

$$m_p = c^{\frac{p+1}{4}} \bmod p.$$

Формула работает только тогда, когда показатель $\frac{p+1}{4}$ – целое число. Если $p \equiv 3 \pmod 4$, то $p + 1$ делится на 4 нацело, и показатель целый. Если бы $p \equiv 1 \pmod 4$, деление дало бы дробь, и понадобился бы вероятностный алгоритм Тонелли–Шенкса.

Второе следствие условия: для $p \equiv 3 \pmod 4$ любой квадратичный вычет имеет ровно два корня, и формула даёт один из них. Вместе для p и q получается $2 \cdot 2 = 4$ корня по модулю n – ровно столько, сколько описывает теория.

> **Проверка формулы на примере**
> Пусть $p = 7$, $c = 15$. Тогда $\frac{p+1}{4} = 2$, и $m_p = 15^2 \bmod 7 = 225 \bmod 7 = 1$. Проверка: $1^2 \bmod 7 = 1$, а $15 \bmod 7 = 1$. Второй корень – это $7 - 1 = 6$, и $6^2 \bmod 7 = 36 \bmod 7 = 1$.

4.3. Почему `p.Bit(1) == 1` равносильно $p \equiv 3 \pmod 4$

`Bit(1)` возвращает второй бит числа, считая с нуля: бит 0 – младший. Остаток по модулю 4 определяется только двумя младшими битами, потому что все старшие биты кратны 4:

$$p \bmod 4 = 2b_1 + b_0,$$

где b_0 – бит 0, b_1 – бит 1.

Для любого простого $p > 2$ число нечётное, значит $b_0 = 1$. Тогда

$$p \bmod 4 = 2b_1 + 1,$$

и остаются два случая:

- $b_1 = 0$: $p \bmod 4 = 1$, то есть $p \equiv 1 \pmod 4$;
- $b_1 = 1$: $p \bmod 4 = 3$, то есть $p \equiv 3 \pmod 4$.

Отсюда `p.Bit(1) == 1` и $p \equiv 3 \pmod 4$ – это одно и то же **для нечётного** p.

> **Тонкость: двойка**
> Для $p = 2$ проверка `p.Bit(1) == 1` тоже истинна: $2 = 10_2$, бит 1 равен единице. Но $2 \bmod 4 = 2$, а не 3, потому что $b_0 = 0$. То есть формально эквивалентность верна только для нечётных чисел. В коде это не опасно: `rand.Prime` вызывается с длиной не меньше 256 бит (`option.Bits / 2` при пресетах 512, 1024 и 2048), поэтому двойка появиться не может.

4.4. Четыре корня и китайская теорема об остатках

Корни по модулю p и по модулю q вычисляются независимо. Коэффициенты Безу y_p и y_q находятся расширенным алгоритмом Евклида:

$$y_p p + y_q q = 1.$$

Из этого следует $y_q q \equiv 1 \pmod p$ и $y_p p \equiv 1 \pmod q$ – каждое слагаемое «работает» по одному модулю и обнуляется по другому. Поэтому два корня по модулю n собираются так:

$$r = \left(m_p y_q q + m_q y_p p\right) \bmod n, \quad s = \left(m_p y_q q - m_q y_p p\right) \bmod n.$$

Знаки m_p и m_q независимы, всего комбинаций четыре. Но r и $-r$ – это один и тот же корень с точностью до знака, и то же для s. Поэтому множество корней записывается коротко:

$$r, \quad n - r, \quad s, \quad n - s.$$

> **Соответствие коду**
> В `Roots` коэффициенты называются `yp` и `yq`, а произведения – `ep = yp*p` и `eq = yq*q`. Тогда `r = mp*eq + mq*ep`, `s = mp*eq - mq*ep`, после чего обе величины приводятся по модулю n. Это в точности формулы выше, только слагаемые переставлены местами.

4.5. Как из четырёх корней выбирается истинный

Каждый корень – это число, которое после расшифрования трактуется как блок фиксированной длины. В блоке есть служебная структура:

- за полезными данными записывается маркер `0x80`;
- остаток блока заполняется нулями;
- последние 8 байт блока занимает метка `MZIRABIN`.

Расшифрование перебирает четыре корня и для каждого проверяет: блок оканчивается меткой, а перед нулевым хвостом стоит `0x80`. Подходящий корень разбирается обратно в полезные данные; если ни один не подошёл, возвращается `ErrNoValidRoot`.

> **Почему это работает и какова вероятность ошибки**
> Посторонний корень – это фактически случайное число по модулю n. Вероятность того, что случайный блок оканчивается фиксированной 8-байтной меткой, равна 2^{-64}, и дополнительно должен угадаться маркер. Поэтому на практике проходит ровно один корень, а совпадение двух практически невозможно. Учебная схема без избыточности такой гарантии не даёт: там получатель обязан выбирать сообщение вручную.

4.6. Зачем блок короче модуля и зачем маркер

Блок имеет длину `k - 1` байт, где `k` – длина n. Это гарантирует $m < n$: иначе число, составленное из `k` байт, могло бы оказаться больше модуля, и возведение в квадрат потеряло бы информацию.

Маркер `0x80` нужен из-за нулевых байтов в конце сообщения. Без него нельзя отличить полезные нули от заполнения: строка `"data"` и строка `"data\0\0\0"` дали бы один и тот же блок. Маркер ставится сразу после данных, поэтому при разборе видно, где кончается сообщение.

> **Что было бы без метки**
> Одной проверки маркера мало: посторонний корень тоже может случайно дать байт `0x80` в подходящем месте. Метка `MZIRABIN` фиксирует восемь байт и снижает вероятность ложного принятия до 2^{-64}.

4.7. Детерминированность и её следствия

В схеме нет случайности на этапе зашифрования: одинаковый блок при одном и том же ключе всегда даёт одинаковый шифртекст. Это свойство учебной схемы, а не недочёт конкретной реализации. Следствия:

- одинаковая полезная нагрузка в разных сообщениях даёт одинаковые блоки шифртекста, поэтому схема не скрывает повторов;
- шифртекст мультипликативен: в схеме без избыточности умножение c на квадрат числа по модулю n даёт шифртекст другого сообщения; наша проверка избыточности такой блок отклонит, но детерминированность остаётся;
- схема детерминирована и потому не является стойкой в смысле IND-CPA.

Для применения к реальным данным нужны рандомизация (случайное заполнение блока) и проверка целостности, например подпись или имитовставка. В работе реализована именно учебная схема из методических указаний.

4.8. Почему $p \neq q$

Если $p = q$, то $n = p^2$, и структура корней другая: у квадратичного вычета по модулю p^2 корней не четыре, а два, а разложение n находится тривиально – достаточно извлечь квадратный корень из n. Поэтому генератор повторяет выбор, пока числа не станут различными, а `validPrivate` отвергает закрытый ключ с одинаковыми простыми.

4.9. Размеры и расширение

Модуль	Блок, байт	Полезных, байт	Служебных, байт
512	63	54	9
1024	127	118	9
2048	255	246	9

Шифртекст каждого блока занимает ровно `k` байт – столько же, сколько модуль. Поэтому сообщение из N блоков даёт Nk байт шифртекста. Например, 300 байт при 512-битном модуле – это шесть блоков по 64 байта, всего 384 байта. Расширение составляет примерно $k/(k-9)$ и для больших модулей стремится к единице.

5. Реализация

5.1. Структура проекта

Файл	Что делает
`internal/crypto/rabin/rabin.go`	примитив: ключи, зашифрование, корни, CRT, избыточность
`internal/crypto/rabin/rabin_test.go`	тесты примитива
`internal/cipher/rabin.go`	адаптер к интерфейсу `AsymCipher`
`internal/ui/asympane`	терминальная панель асимметричного шифрования

5.2. Формат ключей

Открытый ключ – число n, записанное ровно `Bits/8` байт. Закрытый ключ – конкатенация p и q, каждое записано `Bits/16` байт, то есть половиной длины модуля. Фиксированная ширина нужна, чтобы при разборе можно было однозначно разрезать строку пополам.

5.3. Порядок работы дешифратора

1. Проверить закрытый ключ: p и q сравнимы с 3 по модулю 4 и не равны.
2. Вычислить $n = pq$ и проверить, что модуль достаточен для служебных 9 байт.
3. Разбить шифртекст на блоки по `k` байт; отвергнуть неправильную длину.
4. Для каждого блока отвергнуть значения $c \geq n$.
5. Вычислить четыре корня и вернуть первый блок, прошедший проверку метки.
6. Склеить полезные данные блоков в исходное сообщение.

5.4. Интерфейс

Панель `Rabin` показывает открытый и закрытый ключи в шестнадцатеричном виде. `Ctrl+G` генерирует пару выбранного размера, `Ctrl+R` показывает закрытый ключ, `Enter` переключает режим `Encrypt` / `Decrypt`, источник можно задать вручную или выбрать файл. Результат выводится в hex и сохраняется в файл. Механизм кандидатов в панели общий для асимметричных схем, но Рабин возвращает один открытый текст, поэтому переключать варианты не требуется.

6. Проверка работоспособности

Тесты запускаются командой `go test ./internal/crypto/rabin`.

Тест	Что проверяет
`TestLabExampleRoots`	пример $p = 7$, $q = 11$, $n = 77$, $m = 20$: $c = 15$, корни 64, 20, 13, 57
`TestOnlyTrueRootCarriesRedundancy`	из четырёх корней избыточность содержит ровно один – исходный блок
`TestRoundTrip`	512-битный модуль: пустое сообщение, короткая строка, текст на несколько блоков, нулевые байты в конце, случайные двоичные данные
`TestDecryptRejectsBadInput`	отказ при обрезанном шифртексте и при ключе из одинаковых чисел

> **Чего тесты не проверяют**
> Нет сверки с внешними эталонными векторами: для Рабина их нет, поэтому корректность подтверждается примером из методички и круговыми тестами. Круговой тест сам по себе слабее эталонного: он поймает рассогласование шифрования и расшифрования, но не согласованную ошибку в обеих функциях. Поэтому отдельно проверяется, что корни действительно дают c при возведении в квадрат, и что избыточность проходит ровно один корень.

7. Вопросы на защите

? Что делает криптосистема Рабина?

Шифрует сообщение возведением в квадрат по модулю $n = pq$ и расшифровывает его извлечением квадратных корней с помощью закрытого ключа. Стойкость опирается на сложность факторизации n.

? Почему p и q должны быть простыми?

Тогда по модулю каждого из них квадратный корень вычисляется по явной формуле, а по китайской теореме об остатках корни собираются в корни по модулю n. Если числа составные, простой формулы нет, а знание p и q перестаёт быть эквивалентным знанию разложения n.

? Почему $p \equiv q \equiv 3 \pmod 4$?

Чтобы показатель $\frac{p+1}{4}$ в формуле корня был целым. Это же условие гарантирует, что у квадратичного вычета ровно два корня по модулю простого числа.

? Число простое, если остаток от деления на 4 равен 3?

Нет. Остаток и простота – независимые условия. Например, $15 \bmod 4 = 3$, но 15 составное. Сначала число должно быть простым, и дополнительно его остаток по модулю 4 должен быть равен 3.

? Почему `p.Bit(1) == 1` равносильно $p \equiv 3 \pmod 4$?

Остаток по модулю 4 задаётся двумя младшими битами: $p \bmod 4 = 2b_1 + b_0$. Для нечётного p бит b_0 равен 1, поэтому остаток равен 3 ровно тогда, когда $b_1 = 1$. Для двойки эквивалентность нарушается, но двойка не может появиться при длине простого числа от 256 бит.

? Почему у шифртекста четыре корня?

По два корня по модулю p и по модулю q, а китайская теорема об остатках даёт $2 \cdot 2 = 4$ комбинации. Они распадаются на две пары с точностью до знака: r, $n - r$ и s, $n - s$.

? Как получатель узнаёт, какой корень истинный?

По избыточности в блоке: маркеру `0x80` после данных и метке `MZIRABIN` в конце. Проверку проходит ровно один корень, поэтому выбор происходит автоматически.

? Почему блок на байт короче модуля?

Чтобы любое значение блока было строго меньше n. Если взять блок длиной с модуль, число из `k` байт может оказаться больше n, и возведение в квадрат станет необратимым.

? Зачем нужен маркер `0x80`, если есть метка?

Метка подтверждает, что корень правильный, а маркер показывает, где внутри блока кончаются полезные данные. Без маркера завершающие нулевые байты сообщения нельзя отличить от нулевого заполнения блока.

? Что будет при неверном закрытом ключе?

Корни не дадут блока с корректной избыточностью, и дешифратор вернёт `ErrNoValidRoot`. Явной проверки «ключ правильный» нет: избыточность и есть эта проверка.

? Почему нельзя зашифровать блок длиной в модуль?

Можно было бы только при $m < n$, но байтовая запись не гарантирует этого: старший байт может дать значение больше n. Поэтому блок укорачивают на байт, и условие $m < n$ выполняется всегда.

? Чем Рабин отличается от RSA?

В RSA зашифрование – возведение в степень с открытой экспонентой, а стойкость связана с трудностью извлечения корня. В Рабине экспонента фиксирована и равна 2, поэтому зашифрование дешевле, но появляется неоднозначность из четырёх корней, которую нужно снимать избыточностью.

? Детерминированная схема – это плохо?

Для учебной задачи – нет, так проще проверить формулы. Для реального применения – да: одинаковые сообщения дают одинаковый шифртекст, а шифртекст можно предсказуемо менять. Нужны рандомизация и проверка целостности.

? Как проверялась корректность?

Пример из методических указаний: $p = 7$, $q = 11$, $n = 77$, $m = 20$, $c = 15$, корни 64, 20, 13, 57. Проверка, что ровно один корень содержит избыточность. Круговые тесты на 512-битном модуле для разных длин и двоичных данных. Тесты отказов на обрезанном шифртексте и неверном ключе.

? Можно ли применять реализацию к реальным данным?

Нет. Это учебная схема: модуль от 512 до 2048 бит, детерминированное зашифрование, отсутствие рандомизации и проверки целостности. Кроме того, 512-битный модуль факторизуется современными средствами.